

01_variables-datatypes

August 7, 2026

1 01 - PYTHON VARIABLES & DATATYPES

Name: Vamshi / Date: 05/08/2026

- Variable is a name assigned to a value. This name refers to a memory location.
- Python variables are also called as objects.
- Syntax: **variable = value**

variable —> name assigned for memory location & value —> datatype

Rules to define a python variable name:

1. Variable names must ****start with a letter or an underscore (_)****.
2. They **cannot start with a digit/number** but can end with digit/number.
3. It can contain ****letters, digits, and underscores (_)**** but, only if above 2 rules are met.
4. **Cannot contain spaces or special characters.**
5. They are **case-sensitive** (python, Python, and PYTHON are different variables).
6. It **cannot be Python keywords.** =====> Keywords are also known as reserved words.

Comments in Python: - Single line comments- Use **# comment** - Multi line comments- Use “ “ ” **comment “ “ ” or ’ ’ ’ comment ’ ’ ’** or above method can be used for each line

```
[5]: n=10 # Integer value
      n
```

```
[5]: 10
```

```
[6]: 10=n # Not Valid - syntax error - Rule 2
```

```
Cell In[6], line 1
    10=n # Not Valid - syntax error
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='
```

```
[8]: name = 'stark' # String value
      name
```

```
[8]: 'stark'
```

```
[9]: NAME # Name Error - Rule 5, becoz variable NAME is not present (its case_
      ↪sensitive - name and NAME are not the same)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 NAME #

NameError: name 'NAME' is not defined
```

```
[10]: 1a = 25 # Syntax Error - Rule 2
```

```
Cell In[10], line 1
    1a = 25
    ^
SyntaxError: invalid decimal literal
```

```
[11]: name@ = 'tony' # Syntax Error - Rule 4
      name@
```

```
Cell In[11], line 1
    name@ = 'tony' # Name Error - Rule 4
    ^
SyntaxError: invalid syntax
```

```
[12]: name_1 = 'tony' # Rule 3
      name_1
```

```
[12]: 'tony'
```

```
[13]: for = 45 # Syntax Error - Rule 6 (kw cannot be used as variable name)
```

```
Cell In[13], line 1
    for = 45 # Rule 6
    ^
SyntaxError: invalid syntax
```

```
[15]: FOR = 45
      FOR
```

[15]: 45

```
[23]: val = 53.45 # Float value  
val
```

[23]: 53.45

```
[33]: val_1 = True # Boolean value  
val_1
```

[33]: True

To display the output on the screen, print() function is used.

```
[19]: print(FOR)  
print(name_1)  
print(name)  
print(n)
```

45
tony
stark
10

****The type() function is used to identify the datatype of the variable****

```
[35]: print(type(n))  
print(type(name))  
print(type(val))  
print(type(val_1))
```

<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>

To check the list of all keywords and print total number of keywords:

```
[4]: import keyword  
keyword.kwlist # Lists all the key words
```

[4]: ['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',

```

'continue',
'def',
'del',
'elif',
'else',
'except',
'finally',
'for',
'from',
'global',
'if',
'import',
'in',
'is',
'lambda',
'nonlocal',
'not',
'or',
'pass',
'raise',
'return',
'try',
'while',
'with',
'yield']

```

```

[6]: kw = len(keyword.kwlist) # len() function is used get total num of items/length
    ↪ of an object
print("The total number of keywords are", kw)

```

The total number of keywords are 35

```

[17]: first_name = 'VAMSHI'
last_name = ' '
country = 'INDIA'
city = 'Bangalore'
age = 22
is_married = False
skills = ['Python', 'AI', 'Robotics', 'C++', 'Embedded Sys']
person_info = {
    'firstname': 'Mowgli',
    'lastname': ' ',
    'country': 'Jungle',
    'city': 'Near the riverside'
}

# Printing all the Values

```

```

print('First name:', first_name)
print('First name length:', len(first_name))
print('Last name: ', last_name)
print('Last name length: ', len(last_name))
print('Country: ', country)
print('City: ', city)
print('Age: ', age)
print('Married: ', is_married)
print('Skills: ', skills)
print('Person information: ', person_info)

```

```

First name: VAMSHI
First name length: 6
Last name:
Last name length: 1
Country: INDIA
City: Bangalore
Age: 22
Married: False
Skills: ['Python', 'AI', 'Robotics', 'C++', 'Embedded Sys']
Person information: {'firstname': 'Mowgli', 'lastname': ' ', 'country':
'Jungle', 'city': 'Near the riverside'}

```

[18]: *# Declaring multiple variables in one line*

```

first_name, last_name, country, age, is_married = 'vam', 'abc', 'ind', 22, False

print(first_name, last_name, country, age, is_married)
print('First name:', first_name)
print('Last name: ', last_name)
print('Country: ', country)
print('Age: ', age)
print('Married: ', is_married)

```

```

vam abc ind 22 False
First name: vam
Last name: abc
Country: ind
Age: 22
Married: False

```

Name: Vamshi / Date: 06/08/2026 `id()` function returns the address of the memory location

[7]: *# Example 1*

```

a = 101
id(a)

```

[7]: 140721251964472

```
[ ]: # Example 2 --> a1 and a2 have diff values therefore, the address of both the
      ↪ variables is diff
a1, a2 = 102, 201
print(id(a1))
print(id(a2))
```

140721251964472

140721251967672

```
[10]: # Example 3 --> x1 and x2 have same values therefore, both the variables are
      ↪ pointing to same address
x1, x2 = 2, 80
print(id(x1))
print(id(x2))
```

140721251961304

140721251963800

- Functions without arg: fruit()
- Functions with arg: fruits(with seed, without seed) —> it can have multiple arguments

True —> 1 & False —> 0 in boolean operation

```
[12]: b = True      # represents 1
      b1 = False    # represents 0

print(b+b1) # gives 1 as O/P
print(b1/b) # gives 0.0 as O/P
print(b1//b) # gives 0 as O/P
print(b*b1) # gives 0 as O/P
```

1

0.0

0

0

```
[14]: print(b/b1) # Invalid ---> Zero Division Error
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 print(b/b1) # Invalid ---> Zero Division Error

ZeroDivisionError: division by zero
```

Complex Datatype: $a + bj$ form - 'a' is real part - 'b' is imaginary part

The complex data type is used in mathematical, scientific computing, engg. simulations etc....

```
[15]: c = 50 + 30j
      c
```

```
[15]: (50+30j)
```

```
[16]: c1 = 50 + 70j
      c1
```

```
[16]: (50+70j)
```

```
[18]: c+c1 # Adds real part of c with real part of c1 and imaginary part of c with
      ↪imaginary part of c1
```

```
[18]: (100+100j)
```

To get the real and imaginary part:

```
[ ]: print(c)      # complex data
      print(c.real) # real part
      print(c.imag) # imaginary part
```

Name: Vamshi / Date: 07/08/2026 (Strings - Indexing & Slicing)

```
[22]: s = welcome # this is not a string value becoz it must be specified within ' '
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[22], line 1
----> 1 s = welcome # this is not a string value becoz it must be specified
      ↪within ' '

NameError: name 'welcome' is not defined
```

```
[2]: s = welcome here
```

```
Cell In[2], line 1
      s = welcome here
          ^
SyntaxError: invalid syntax
```

```
[3]: s = 'welcome'
      s
```

```
[3]: 'welcome'
```

```
[4]: s1 = "welcome"
s1
```

```
[4]: 'welcome'
```

```
[5]: s2 = '''welcome'''
s2
```

```
[5]: 'welcome'
```

```
[7]: s3 = 'welcome to datatypes of python
      I am learning strings now'
s3
```

```
Cell In[7], line 1
      s3 = 'welcome to datatypes of python
            ~
SyntaxError: unterminated string literal (detected at line 1)
```

- We make use of ''' ''' for a multi line string data

```
[11]: s4 = '''welcome to datatypes of python
          I am learning
          strings now'''
s4
```

```
[11]: 'welcome to datatypes of python \n I am learning \n strings now'
```

String Indexing

- Types:
 - Forward Indexing
 - Backward Indexing
 - Step Indexing

```
[12]: st = 'robots'
st
```

```
[12]: 'robots'
```


2 [0 1 2 3 4 5] —> This is Forward Index (left to right)

3 ['r' 'o' 'b' 'o' 't' 's']

4 [-6 -5 -4 -3 -2 -1] —> This is Backward Index (right to left)

Note: Python Index always starts with 0

```
[13]: st[0]
```

```
[13]: 'r'
```

```
[25]: st[1]
```

```
[25]: 'o'
```

```
[26]: st[2]
```

```
[26]: 'b'
```

```
[24]: st[3]
```

```
[24]: 'o'
```

```
[27]: st[4]
```

```
[27]: 't'
```

```
[14]: st[5]
```

```
[14]: 's'
```

```
[23]: st[6] # Not valid becoz the index is not present
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[23], line 1  
----> 1 st[6] # Not valid becoz the index is not present  
  
IndexError: string index out of range
```

String Slicing is used to slice or get a specific part of a string.

```
[28]: string = 'This is a test code'  
      string[:] # it returns the entire string
```

```
[28]: 'This is a test code'
```

```
[29]: string[0:4] # it returns from index 0 to index 3 ---> (n-1) where 'n' is the
      ↪ stop index which is excluded
```

```
[29]: 'This'
```

```
[30]: string[4:] # it starts from index 4 and gives the entire string till the end
      ↪ becoz stop index is not specified
```

```
[30]: ' is a test code'
```

```
[31]: string[:10] # start index not specified so it starts from zero and gives the 0/
      ↪ P till (n-1) i.e 9
```

```
[31]: 'This is a '
```

```
[33]: len(string) # length of the string
```

```
[33]: 19
```

```
[38]: stg1 = 'Hooray!!!! '
      print(stg1)

      stg1 = stg1 * 10 # multiplies stg1 10 times
      print(stg1)
```

```
Hooray!!!!
```

```
Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!! Hooray!!!!
```

```
Hooray!!!! Hooray!!!! Hooray!!!!
```

```
[39]: # String Concatenation

      n1 = 'Hi, '
      n2 = 'This is Bob.'
      n1+n2 # It combines both the strings together
```

```
[39]: 'Hi, This is Bob.'
```